

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

Homework 8

NOTE: You are not required to hand in the following exercises, but you are strongly encouraged to complete them to strengthen your understanding of the concepts covered in class.

1. Write a wrapper function for `sleep`, called `snooze`, with the following interface:

```
unsigned int snooze(unsigned int secs);
```

The `snooze` function behaves exactly as the `sleep` function, except that it prints a message describing how long the process actually slept:

```
Slept for 4 of 5 secs.
```

The `sleep` function returns zero if the requested amount of time has elapsed, and the number of seconds still left to sleep otherwise. The latter case is possible if the `sleep` function returns prematurely because it was interrupted by a signal (Exercise 2).

2. Write a program, called `snooze`, that takes a single command line argument, calls the `snooze` function from Exercise 1 with this argument, and then terminates. Write your program so that the user can interrupt the `snooze` function by typing `ctrl-c` at the keyboard. For example:

```
$ ./snooze 5
Slept for 3 of 5 secs.    User hits ctrl-c after 3 seconds
$
```

3. What is the output of the following program?

```
pid_t pid;
int counter = 2;

void handler1(int sig) {
    counter = counter - 1;
    printf("%d", counter);
    fflush(stdout);
    exit(0);
}

int main() {
    signal(SIGUSR1, handler1);

    printf("%d", counter);
    fflush(stdout);
```

```

    if ((pid = fork()) == 0) {
        while(1) {};
    }
    kill(pid, SIGUSR1);
    waitpid(-1, NULL, 0);
    counter = counter + 1;
    printf("%d", counter);
    exit(0);
}

```

where SIGUSR1 is one of the two user-defined signals reserved by the POSIX standard for custom application-level notifications and synchronization; the other is SIGUSR2. Unlike signals such as SIGKILL or SIGSEGV, which have predefined meanings, the kernel does not attach any specific meaning to user-defined signals. Their meaning is entirely defined by the logic you write in your program.

4. One of your classmates is thinking of using signals to allow a parent process to count events that occur in a child process. The idea is to notify the parent each time an event occurs by sending it a signal, and letting the parent's signal handler increment a global counter variable, which the parent can then inspect after the child has terminated. However, when he runs the test program below on his system, he discovers that when the parent calls `printf`, `counter` always has a value of 2, even though the child has sent five signals to the parent. Perplexed, he comes to you for help. Can you explain the bug?

```

int counter = 0;

void handler(int sig)
{
    counter++;
    sleep(1); /* Do some work in the handler */
    return;
}

int main()
{
    int i;

    signal(SIGUSR2, handler);

    if (fork() == 0) { /* Child */
        for (i = 0; i < 5; i++) {
            kill(getppid(), SIGUSR2);
            printf("sent SIGUSR2 to parent\n");
        }
        exit(0);
    }

    wait(NULL);
    printf("counter=%d\n", counter);
    exit(0);
}

```

- **Welcome: Sean**

- [LogOut](#)

